ЛАБОРАТОРНАЯ РАБОТА № 8

«Введение в работу с XML, JSON, HTML и Excel»

Группа: РИМ-150950
Студент: Ляхова Елизавета Олеговна

Ссылка на github: https://github.com/Lizaswer/java_course_master.git

Цель: получить представление о механизме многопоточности в языке программирования Java

Ход работы:

1. XML парсер

XmlCreateExample.java

```
package lr8.xml;

import javax.xml.parsers.*;
import javax.xml.transform.*;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;
import org.w3c.dom.*;
import java.io.File;

public class XmlCreateExample {
    public static void main(String[] args) {
        try {
            DocumentBuilderFactory docFactory =
DocumentBuilderFactory.newInstance();
            DocumentBuilder docBuilder =
docFactory.newDocumentBuilder();
            Document doc = docBuilder.newDocument();

            Element rootElement =
doc.createElement("library");
            doc.appendChild(rootElement);

            // Книга 1
            Element book1 = doc.createElement("book");
            rootElement.appendChild(book1);

            Element title1 =
doc.createElement("title");

title1.appendChild(doc.createTextNode("Война и
мир"));
```

```
            book1.appendChild(title1);

            Element author1 =
doc.createElement("author");

author1.appendChild(doc.createTextNode("Лев
Толстой"));
            book1.appendChild(author1);

            Element year1 = doc.createElement("year");

year1.appendChild(doc.createTextNode("1869"));
            book1.appendChild(year1);

            // Книга 2
            Element book2 = doc.createElement("book");
            rootElement.appendChild(book2);

            Element title2 =
doc.createElement("title");

title2.appendChild(doc.createTextNode("Мастер и
Маргарита"));
            book2.appendChild(title2);

            Element author2 =
doc.createElement("author");

author2.appendChild(doc.createTextNode("Михаил
Булгаков"));
            book2.appendChild(author2);

            Element year2 = doc.createElement("year");

year2.appendChild(doc.createTextNode("1967"));
            book2.appendChild(year2);

            // Сохранение
```

```
            TransformerFactory transformerFactory =
TransformerFactory.newInstance();
            Transformer transformer =
transformerFactory.newTransformer();

transformer.setOutputProperty(OutputKeys.INDENT,
"yes");
            DOMSource source = new DOMSource(doc);
            StreamResult result = new StreamResult(new
File("src/lr8/recources/library.xml"));
            transformer.transform(source, result);

            System.out.println("✅ XML файл создан:
src/lr8/recources/library.xml");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

XmlReadExample. java

```
package lr8.xml;

import javax.xml.parsers.*;
import org.w3c.dom.*;
import java.io.File;

public class XmlReadExample {
    public static void main(String[] args) {
        try {
            File inputFile = new
File("src/lr8/recources/library.xml");
            DocumentBuilderFactory dbFactory =
DocumentBuilderFactory.newInstance();
```

```
            DocumentBuilder dBuilder =
dbFactory.newDocumentBuilder();
            Document doc = dBuilder.parse(inputFile);
            doc.getDocumentElement().normalize();

            System.out.println("Корневой элемент: " +
doc.getDocumentElement().getNodeName());

            NodeList nList =
doc.getElementsByTagName("book");

            for (int i = 0; i < nList.getLength();
i++) {
                Node nNode = nList.item(i);

                if (nNode.getNodeType() ==
Node.ELEMENT_NODE) {
                    Element eElement = (Element)
nNode;

                    String title =
eElement.getElementsByTagName("title").item(0).getTe
xtContent();
                    String author =
eElement.getElementsByTagName("author").item(0).getT
extContent();
                    String year =
eElement.getElementsByTagName("year").item(0).getTex
tContent();

                    System.out.println("\n📚 Книга " +
(i+1) + ":");
                    System.out.println("   Название: "
+ title);
                    System.out.println("   Автор: " +
author);
                    System.out.println("   Год: " +
year);
```

```
                }
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
```

XmlParserComplete.java

```
package lr8.xml;

import org.w3c.dom.*;
import javax.xml.parsers.*;
import javax.xml.transform.*;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;
import java.io.File;
import java.util.Scanner;

public class XmlParserComplete {
    private static final String FILE_PATH =
"src/lr8/recources/library.xml";

    public static void main(String[] args) {
        createInitialXml();
        Scanner scanner = new Scanner(System.in);

        while (true) {
            System.out.println("XML ПАРСЕР - МЕНЮ");
            System.out.println("1. Показать все
книги");
            System.out.println("2. Добавить новую
книгу");
            System.out.println("3. Поиск книг по
автору");
```

```
            System.out.println("4. Поиск книг по
году");
            System.out.println("5. Удалить книгу по
названию");
            System.out.println("6. Выход");
            System.out.print("\nВыберите действие
(1-6): ");

            int choice = scanner.nextInt();
            scanner.nextLine();

            switch (choice) {
                case 1 -> showAllBooks();
                case 2 -> addBook(scanner);
                case 3 -> searchByAuthor(scanner);
                case 4 -> searchByYear(scanner);
                case 5 -> deleteBookByTitle(scanner);
                case 6 -> {
                    System.out.println("До
свидания!");
                    return;
                }
                default ->
System.out.println("Неверный выбор!");
            }
        }
    }

    private static void createInitialXml() {
        File file = new File(FILE_PATH);
        if (file.exists()) return;

        try {
            DocumentBuilderFactory docFactory =
DocumentBuilderFactory.newInstance();
            DocumentBuilder docBuilder =
docFactory.newDocumentBuilder();
            Document doc = docBuilder.newDocument();
```

```
            Element rootElement =
doc.createElement("library");
            doc.appendChild(rootElement);

            String[][] defaultBooks = {
                    {"Война и мир", "Лев Толстой",
"1869"},
                    {"Мастер и Маргарита", "Михаил
Булгаков", "1967"},
                    {"Преступление и наказание",
"Фёдор Достоевский", "1866"}
            };

            for (String[] bookData : defaultBooks) {
                Element book =
doc.createElement("book");

                Element title =
doc.createElement("title");

title.appendChild(doc.createTextNode(bookData[0]));
                book.appendChild(title);

                Element author =
doc.createElement("author");

author.appendChild(doc.createTextNode(bookData[1]));
                book.appendChild(author);

                Element year =
doc.createElement("year");

year.appendChild(doc.createTextNode(bookData[2]));
                book.appendChild(year);

                rootElement.appendChild(book);
            }
```

```
            saveDocument(doc);
            System.out.println("✅ Создан файл
library.xml");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    private static void saveDocument(Document doc)
throws TransformerException {
        TransformerFactory transformerFactory =
TransformerFactory.newInstance();
        Transformer transformer =
transformerFactory.newTransformer();

transformer.setOutputProperty(OutputKeys.INDENT,
"yes");
        DOMSource source = new DOMSource(doc);
        StreamResult result = new StreamResult(new
File(FILE_PATH));
        transformer.transform(source, result);
    }

    private static Document loadDocument() throws
Exception {
        DocumentBuilderFactory factory =
DocumentBuilderFactory.newInstance();
        DocumentBuilder builder =
factory.newDocumentBuilder();
        return builder.parse(new File(FILE_PATH));
    }

    private static String getElementText(Element
parent, String tagName) {
        NodeList nodes =
parent.getElementsByTagName(tagName);
```

```
        return nodes.getLength() > 0 ?
nodes.item(0).getTextContent() : "";
    }

    private static void showAllBooks() {
        try {
            Document doc = loadDocument();
            NodeList books =
doc.getElementsByTagName("book");

            if (books.getLength() == 0) {
                System.out.println("Библиотека
пуста.");
                return;
            }

            System.out.println("\n=== СПИСОК ВСЕХ КНИГ
===");
            for (int i = 0; i < books.getLength();
i++) {
                Element book = (Element)
books.item(i);
                System.out.printf("%d. \"%s\"%n", i +
1, getElementText(book, "title"));
                System.out.printf("   Автор: %s%n",
getElementText(book, "author"));
                System.out.printf("   Год: %s%n",
getElementText(book, "year"));

System.out.println("----------------------------------
-------");
            }
        } catch (Exception e) {
            System.err.println("Ошибка: " +
e.getMessage());
        }
    }
```

```
    private static void addBook(Scanner scanner) {
        try {
            System.out.print("Название книги: ");
            String title = scanner.nextLine();
            System.out.print("Автор: ");
            String author = scanner.nextLine();
            System.out.print("Год издания: ");
            String year = scanner.nextLine();

            Document doc = loadDocument();
            Element library =
doc.getDocumentElement();
            Element book = doc.createElement("book");

            Element titleElem =
doc.createElement("title");
            titleElem.setTextContent(title);
            book.appendChild(titleElem);

            Element authorElem =
doc.createElement("author");
            authorElem.setTextContent(author);
            book.appendChild(authorElem);

            Element yearElem =
doc.createElement("year");
            yearElem.setTextContent(year);
            book.appendChild(yearElem);

            library.appendChild(book);
            saveDocument(doc);

            System.out.println("Книга \"" + title +
"\" добавлена!");
        } catch (Exception e) {
            System.err.println("Ошибка: " +
e.getMessage());
        }
```

```
    }

    private static void searchByAuthor(Scanner
scanner) {
        try {
            System.out.print("Введите автора: ");
            String searchAuthor = scanner.nextLine();

            Document doc = loadDocument();
            NodeList books =
doc.getElementsByTagName("book");

            boolean found = false;
            System.out.println("\n=== РЕЗУЛЬТАТЫ
ПОИСКА ===");

            for (int i = 0; i < books.getLength();
i++) {
                Element book = (Element)
books.item(i);
                String author = getElementText(book,
"author");
                if
(author.equalsIgnoreCase(searchAuthor)) {
                    System.out.printf("\"%s\" (%s)%n",
                            getElementText(book,
"title"),
                            getElementText(book,
"year"));
                    found = true;
                }
            }
            if (!found) System.out.println("Книги не
найдены.");
        } catch (Exception e) {
            System.err.println("Ошибка: " +
e.getMessage());
        }
```

```
    }

    private static void searchByYear(Scanner scanner)
{
        try {
            System.out.print("Введите год: ");
            String searchYear = scanner.nextLine();

            Document doc = loadDocument();
            NodeList books =
doc.getElementsByTagName("book");

            boolean found = false;
            System.out.println("\n=== РЕЗУЛЬТАТЫ
ПОИСКА ===");

            for (int i = 0; i < books.getLength();
i++) {
                Element book = (Element)
books.item(i);
                String year = getElementText(book,
"year");
                if (year.equals(searchYear)) {
                    System.out.printf("\"%s\" - %s%n",
                            getElementText(book,
"title"),
                            getElementText(book,
"author"));
                    found = true;
                }
            }
            if (!found) System.out.println("Книги не
найдены.");
        } catch (Exception e) {
            System.err.println("Ошибка: " +
e.getMessage());
        }
    }
```

```
   private static void deleteBookByTitle(Scanner
scanner) {
        try {
            System.out.print("Введите название книги
для удаления: ");
            String deleteTitle = scanner.nextLine();

            Document doc = loadDocument();
            NodeList books =
doc.getElementsByTagName("book");

            Element bookToDelete = null;
            for (int i = 0; i < books.getLength();
i++) {
                Element book = (Element)
books.item(i);
                if (getElementText(book,
"title").equalsIgnoreCase(deleteTitle)) {
                    bookToDelete = book;
                    break;
                }
            }

            if (bookToDelete == null) {
                System.out.println("Книга не
найдена.");
                return;
            }


bookToDelete.getParentNode().removeChild(bookToDelet
e);
            saveDocument(doc);
            System.out.println("Книга \"" +
deleteTitle + "\" удалена.");
        } catch (Exception e) {
```

```
            System.err.println("Ошибка: " +
e.getMessage());
        }
    }
}
```

2. JSON парсер

JsonCreateExample.java

```
package lr8.json;

import org.json.simple.JSONArray;
import org.json.simple.JSONObject;
import java.io.FileWriter;
import java.io.IOException;

public class JsonCreateExample {
    @SuppressWarnings("unchecked")
public static void main(String[] args) {
    JSONObject library = new JSONObject();
    JSONArray books = new JSONArray();

    JSONObject book1 = new JSONObject();
book1.put("title", "Java Programming");
    book1.put("author", "John Doe");
    book1.put("year", "2015");
    books.add(book1);

    JSONObject book2 = new JSONObject();
book2.put("title", "Python Programming");
    book2.put("author", "Jane Smith");
    book2.put("year", "2018");
    books.add(book2);

    library.put("books", books);
```

```
            try (FileWriter file = new
  FileWriter("src/Ir8/recources/books.json")) {
            file.write(library.toJSONString());
             System.out.println("✅ JSON файл
    создан: src/Ir8/recources/books.json");
            } catch (IOException e) {
                e.printStackTrace();
                    }
                }
            }
```

JsonReadExample.java

```
package lr8.json;

import org.json.simple.JSONArray;
import org.json.simple.JSONObject;
import org.json.simple.parser.JSONParser;
import java.io.FileReader;

public class JsonReadExample {
   public static void main(String[] args) {
       JSONParser parser = new JSONParser();

       try (FileReader reader = new
FileReader("src/lr8/recources/books.json")) {
           JSONObject obj = (JSONObject)
parser.parse(reader);
           JSONArray books = (JSONArray)
obj.get("books");

           System.out.println("=== СПИСОК КНИГ
===");
           for (Object bookObj : books) {
               JSONObject book = (JSONObject)
bookObj;
```

```
                System.out.println("📚 " +
book.get("title"));
                System.out.println("    ✍️ " +
book.get("author"));
                System.out.println("    📅 " +
book.get("year"));

System.out.println("----------------------------
--");
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

JsonParserComplete.java

```
package lr8.json;

import org.json.simple.JSONArray;
import org.json.simple.JSONObject;
import org.json.simple.parser.JSONParser;
import java.io.FileReader;
import java.io.FileWriter;
import java.util.Iterator;
import java.util.Scanner;

public class JsonParserComplete {
    private static final String FILE_PATH =
"src/Ir8/recources/books.json";

    @SuppressWarnings("unchecked")
    public static void main(String[] args) {
        createInitialJson();
        Scanner scanner = new Scanner(System.in);
```

```
        while (true) {
            System.out.println("          JSON
ПАРСЕР - МЕНЮ");
            System.out.println("1. Показать все
книги");
            System.out.println("2. Добавить новую
книгу");
            System.out.println("3. Поиск книг по
автору");
            System.out.println("4. Удалить книгу
по названию");
            System.out.println("5. Выход");
            System.out.print("\nВыберите действие
(1-5): ");

            int choice = scanner.nextInt();
            scanner.nextLine();

            switch (choice) {
                case 1 -> showAllBooks();
                case 2 -> addBook(scanner);
                case 3 ->
searchByAuthor(scanner);
                case 4 ->
deleteBookByTitle(scanner);
                case 5 -> {
                    System.out.println("До
свидания!");
                    return;
                }
                default ->
System.out.println("Неверный выбор!");
            }
        }
    }

    @SuppressWarnings("unchecked")
    private static void createInitialJson() {
```

```
        java.io.File file = new
java.io.File(FILE_PATH);
        if (file.exists()) return;

        JSONObject library = new JSONObject();
        JSONArray books = new JSONArray();

        String[][] defaultBooks = {
                {"Java Programming", "John Doe",
"2015"},
                {"Python Programming", "Jane
Smith", "2018"},
                {"Clean Code", "Robert Martin",
"2008"}
        };

        for (String[] bookData : defaultBooks) {
            JSONObject book = new JSONObject();
            book.put("title", bookData[0]);
            book.put("author", bookData[1]);
            book.put("year", bookData[2]);
            books.add(book);
        }

        library.put("books", books);

        try (FileWriter writer = new
FileWriter(FILE_PATH)) {
            writer.write(library.toJSONString());
            System.out.println("✅ Создан файл
books.json");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    private static JSONObject loadJson() throws
Exception {
```

```
        JSONParser parser = new JSONParser();
        try (FileReader reader = new
FileReader(FILE_PATH)) {
            return (JSONObject)
parser.parse(reader);
        }
    }

    private static void saveJson(JSONObject
jsonObject) throws Exception {
        try (FileWriter writer = new
FileWriter(FILE_PATH)) {

writer.write(jsonObject.toJSONString());
        }
    }

    private static void showAllBooks() {
        try {
            JSONObject jsonObject = loadJson();
            JSONArray books = (JSONArray)
jsonObject.get("books");

            if (books.isEmpty()) {
                System.out.println("Библиотека
пуста.");
                return;
            }

            System.out.println("\n=== СПИСОК ВСЕХ
КНИГ ===");
            for (int i = 0; i < books.size();
i++) {
                JSONObject book = (JSONObject)
books.get(i);
                System.out.printf("%d. \"%s\"%n",
i + 1, book.get("title"));
```

```
                System.out.printf("   Автор:
%s%n", book.get("author"));
                System.out.printf("   Год: %s%n",
book.get("year"));

System.out.println("----------------------------
------------");
            }
        } catch (Exception e) {
            System.err.println("Ошибка: " +
e.getMessage());
        }
    }

    @SuppressWarnings("unchecked")
    private static void addBook(Scanner scanner)
{
        try {
            System.out.print("Название книги: ");
            String title = scanner.nextLine();
            System.out.print("Автор: ");
            String author = scanner.nextLine();
            System.out.print("Год издания: ");
            String year = scanner.nextLine();

            JSONObject jsonObject = loadJson();
            JSONArray books = (JSONArray)
jsonObject.get("books");

            JSONObject newBook = new
JSONObject();
            newBook.put("title", title);
            newBook.put("author", author);
            newBook.put("year", year);

            books.add(newBook);
            saveJson(jsonObject);
```

```
            System.out.println("Книга \"" + title
+ "\" добавлена!");
        } catch (Exception e) {
            System.err.println("Ошибка: " +
e.getMessage());
        }
    }

    private static void searchByAuthor(Scanner
scanner) {
        try {
            System.out.print("Введите автора: ");
            String searchAuthor =
scanner.nextLine();

            JSONObject jsonObject = loadJson();
            JSONArray books = (JSONArray)
jsonObject.get("books");

            boolean found = false;
            System.out.println("\n=== РЕЗУЛЬТАТЫ
ПОИСКА ===");

            for (Object obj : books) {
                JSONObject book = (JSONObject)
obj;
                if
(book.get("author").toString().equalsIgnoreCase(
searchAuthor)) {
                    System.out.printf("\"%s\"
(%s)%n", book.get("title"), book.get("year"));
                    found = true;
                }
            }
            if (!found) System.out.println("Книги
не найдены.");
        } catch (Exception e) {
```

```
            System.err.println("Ошибка: " +
e.getMessage());
        }
    }

    private static void deleteBookByTitle(Scanner
scanner) {
        try {
            System.out.print("Введите название
книги для удаления: ");
            String deleteTitle =
scanner.nextLine();

            JSONObject jsonObject = loadJson();
            JSONArray books = (JSONArray)
jsonObject.get("books");

            Iterator<Object> iterator =
books.iterator();
            boolean found = false;

            while (iterator.hasNext()) {
                JSONObject book = (JSONObject)
iterator.next();
                if
(book.get("title").toString().equalsIgnoreCase(d
eleteTitle)) {
                    iterator.remove();
                    found = true;
                    break;
                }
            }

            if (!found) {
                System.out.println("Книга не
найдена.");
                return;
            }
```

```
            saveJson(jsonObject);
            System.out.println("Книга \"" +
deleteTitle + "\" удалена.");
        } catch (Exception e) {
            System.err.println("Ошибка: " +
e.getMessage());
        }
    }
}
```

3.HTML парсер

HtmlLinkParserExample.java

```
package lr8.html;

import org.jsoup.Jsoup;
import org.jsoup.nodes.Document;
import org.jsoup.nodes.Element;
import org.jsoup.select.Elements;
import java.io.IOException;

public class HtmlLinkParserExample {
    public static void main(String[] args) {
        try {
            Document doc =
Jsoup.connect("https://itlearn.ru/first-steps").
get();
            Elements links =
doc.select("a[href]");

            System.out.println("=== НАЙДЕННЫЕ
ССЫЛКИ ===");
            int count = 0;
            for (Element link : links) {
                String url =
link.attr("abs:href");
                String text = link.text();
```

```
                if (!text.isEmpty()) {
                    count++;
                    System.out.println(count + ".
" + text);
                    System.out.println("   → " +
url);
                }
            }
            System.out.println("\n✅ Всего
найдено ссылок: " + count);

        } catch (IOException e) {
            System.err.println("❌ Ошибка
подключения: " + e.getMessage());
        }
    }
}
```

HtmlNewsParserExample.java

```
package lr8.html;

import org.jsoup.Jsoup;
import org.jsoup.nodes.Document;
import org.jsoup.nodes.Element;
import org.jsoup.select.Elements;
import java.io.IOException;

public class HtmlNewsParserExample {
    public static void main(String[] args) {
        try {
            Document doc =
Jsoup.connect("http://fat.urfu.ru/index.html").g
et();

            Elements textElements =
doc.select("p, div, td");
```

```
            System.out.println("=== НОВОСТИ САЙТА
===");
            int newsCount = 0;

            for (Element element : textElements)
{
                String text =
element.text().trim();
                if (text.length() > 50 &&
text.length() < 500 && !text.contains("©")) {
                    newsCount++;
                    System.out.println("\n📰
Новость #" + newsCount);

System.out.println(text.substring(0,
Math.min(150, text.length())) + "...");
                    if (newsCount >= 10) break;
                }
            }

            if (newsCount == 0) {
                System.out.println("Новости не
найдены.");
            }

        } catch (IOException e) {
            System.err.println("Ошибка
подключения: " + e.getMessage());
        }
    }
}
```

HtmlParserComplete.java

```
package lr8.html;

import org.jsoup.Jsoup;
import org.jsoup.nodes.Document;
import org.jsoup.nodes.Element;
import org.jsoup.select.Elements;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class HtmlParserComplete {
    private static final String URL =
"http://fat.urfu.ru/index.html";
    private static final String OUTPUT_FILE =
"src/Ir8/recources/news_output.txt";
    private static final int MAX_ATTEMPTS = 3;

    public static void main(String[] args) {

        System.out.println(" HTML ПАРСЕР
НОВОСТЕЙ");

        parseAndSaveNews();
    }

    private static void parseAndSaveNews() {
        for (int attempt = 1; attempt <=
MAX_ATTEMPTS; attempt++) {
            System.out.println("\nПопытка " +
attempt + " из " + MAX_ATTEMPTS);
```

```
            try {
                Document doc = Jsoup.connect(URL)
                        .userAgent("Mozilla/5.0
(Windows NT 10.0; Win64; x64)")
                        .timeout(10000)
                        .get();

                System.out.println("Страница
загружена!");
                saveNewsToFile(doc);
                System.out.println("\nНовости
сохранены в " + OUTPUT_FILE);
                return;

            } catch (IOException e) {
                System.err.println("Ошибка: " +
e.getMessage());
                if (attempt == MAX_ATTEMPTS) {
                    System.err.println("Не
удалось загрузить страницу.");
                } else {
                    System.out.println("Повтор
через 3 секунды...");
                    try { Thread.sleep(3000); }
catch (InterruptedException ie) {}
                }
            }
        }
    }

    private static void saveNewsToFile(Document
doc) {
        try (PrintWriter writer = new
PrintWriter(new FileWriter(OUTPUT_FILE))) {
            String timestamp =
LocalDateTime.now().format(DateTimeFormatter.ofPa
ttern("dd.MM.yyyy HH:mm:ss"));
```

```
            writer.println("НОВОСТИ С САЙТА: " +
URL);
            writer.println("Дата: " + timestamp);
            writer.println();

            Elements textElements =
doc.select("p, div, td");
            int newsCount = 0;

            for (Element element : textElements)
{
                String text =
element.text().trim();
                if (isNewsText(text) && newsCount
< 20) {
                    newsCount++;
                    writer.println("НОВОСТЬ #" +
newsCount);
                    writer.println(text);

writer.println("-------------------------------
--------");
                    writer.println();

                    System.out.println("\nНовость
#" + newsCount);

System.out.println(text.substring(0,
Math.min(100, text.length())) + "...");
                }
            }

            writer.println("\nВсего найдено
новостей: " + newsCount);
            System.out.println("\nВсего новостей:
" + newsCount);

        } catch (IOException e) {
```

```
            System.err.println("Ошибка записи: "
+ e.getMessage());
        }
    }

    private static boolean isNewsText(String
text) {
        if (text == null || text.isEmpty())
return false;
        if (text.length() < 30) return false;

        String[] exclude = {"©", "карта сайта",
"email", "телефон", "поиск"};
        for (String word : exclude) {
            if
(text.toLowerCase().contains(word)) return
false;
        }
        return true;
    }
}
```

4.Excel парсер

ExcelCreateExample.java

```
package lr8.excel;

import org.apache.poi.ss.usermodel.*;
import
org.apache.poi.xssf.usermodel.XSSFWorkbook;
import java.io.FileOutputStream;
import java.io.IOException;

public class ExcelCreateExample {
public static void main(String[] args) {
```

```
            try (Workbook workbook = new
              XSSFWorkbook()) {
                      Sheet sheet =
        workbook.createSheet("Товары");

                      // Заголовки
           Row headerRow = sheet.createRow(0);

 headerRow.createCell(0).setCellValue("Название
                    товара");

headerRow.createCell(1).setCellValue("Характерис
                    тики");

headerRow.createCell(2).setCellValue("Стоимость"
                      );

                        // Данные
             Row row1 = sheet.createRow(1);

 row1.createCell(0).setCellValue("Книга Java");
           row1.createCell(1).setCellValue("512
                    стр.");

    row1.createCell(2).setCellValue(1500);

             Row row2 = sheet.createRow(2);

  row2.createCell(0).setCellValue("Ноутбук");
          row2.createCell(1).setCellValue("16GB
                    RAM");

    row2.createCell(2).setCellValue(45000);

               for (int i = 0; i < 3; i++)
          sheet.autoSizeColumn(i);
```

```
            try (FileOutputStream out = new
FileOutputStream("src/Ir8/recources/data.xlsx"))
                    {
                workbook.write(out);
            System.out.println(" Excel файл
    создан: src/Ir8/recources/data.xlsx");
                        }
            } catch (IOException e) {
                e.printStackTrace();
                    }
                }
            }
```

ExcelReadExample.java

```
package lr8.excel;

import org.apache.poi.ss.usermodel.*;
import
org.apache.poi.xssf.usermodel.XSSFWorkbook;
import java.io.FileInputStream;
import java.io.IOException;

public class ExcelReadExample {
   public static void main(String[] args) {
       try (FileInputStream file = new
FileInputStream("src/Ir8/recources/data.xlsx");
            Workbook workbook = new
XSSFWorkbook(file)) {

           Sheet sheet =
workbook.getSheet("Товары");
           if (sheet == null) {
               System.out.println("Лист 'Товары'
не найден!");
               return;
           }
```

```
            System.out.println("=== СОДЕРЖИМОЕ
EXCEL ===");
            for (Row row : sheet) {
                for (Cell cell : row) {
                    switch (cell.getCellType()) {
                        case STRING ->
System.out.print(cell.getStringCellValue() +
"\t");
                        case NUMERIC ->
System.out.print((long)
cell.getNumericCellValue() + "\t");
                        default ->
System.out.print(" \t");
                    }
                }
                System.out.println();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

ExcelParserComplete.java

```
package lr8.excel;

import org.apache.poi.ss.usermodel.*;
import
org.apache.poi.xssf.usermodel.XSSFWorkbook;
import
org.apache.poi.hssf.usermodel.HSSFWorkbook;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.util.Scanner;
```

```java
public class ExcelParserComplete {
    private static final String FILE_PATH =
"src/Ir8/recources/data.xlsx";

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        while (true) {
            System.out.println("          EXCEL
ПАРСЕР - МЕНЮ");
            System.out.println("1. Прочитать
Excel файл");
            System.out.println("2. Создать Excel
файл");
            System.out.println("3. Выход");
            System.out.print("Выберите действие:
");

            int choice = scanner.nextInt();

            switch (choice) {
                case 1 -> readExcelFile();
                case 2 -> createExcelFile();
                case 3 -> {
                    System.out.println("До
свидания!");
                    return;
                }
                default ->
System.out.println("Неверный выбор!");
            }
        }
    }

    private static void readExcelFile() {
        File file = new File(FILE_PATH);

        if (!file.exists()) {
```

```
            System.err.println("Файл не
существует! Сначала создайте его (пункт 2)");
            return;
        }

        try (FileInputStream fis = new
FileInputStream(file)) {
            Workbook workbook;
            if (FILE_PATH.endsWith(".xlsx")) {
                workbook = new XSSFWorkbook(fis);
            } else {
                workbook = new HSSFWorkbook(fis);
            }

            if (workbook.getNumberOfSheets() ==
0) {
                System.err.println("Нет листов в
файле!");
                return;
            }

            Sheet sheet = workbook.getSheetAt(0);
            System.out.println("\n=== СОДЕРЖИМОЕ
ЛИСТА: " + sheet.getSheetName() + " ===");

            boolean hasData = false;
            for (Row row : sheet) {
                for (Cell cell : row) {
                    String value =
getCellValue(cell);
                    System.out.print(value +
"\t");
                    hasData = true;
                }
                System.out.println();
            }
```

```
            if (!hasData)
System.out.println("Файл пуст!");

            workbook.close();

        } catch (Exception e) {
            System.err.println("Ошибка чтения: "
+ e.getMessage());
        }
    }

    private static void createExcelFile() {
        try (Workbook workbook = new
XSSFWorkbook()) {
            Sheet sheet =
workbook.createSheet("Данные");

            Row header = sheet.createRow(0);
            String[] headers = {"ID", "Имя",
"Возраст", "Город"};
            for (int i = 0; i < headers.length;
i++) {

header.createCell(i).setCellValue(headers[i]);
            }

            Object[][] data = {
                    {1, "Иван Иванов", 25,
"Москва"},
                    {2, "Петр Петров", 30,
"СПб"},
                    {3, "Мария Сидорова", 28,
"Екб"}
            };

            int rowNum = 1;
            for (Object[] rowData : data) {
```

```
                Row row =
sheet.createRow(rowNum++);

row.createCell(0).setCellValue((Integer)
rowData[0]);

row.createCell(1).setCellValue((String)
rowData[1]);

row.createCell(2).setCellValue((Integer)
rowData[2]);

row.createCell(3).setCellValue((String)
rowData[3]);
            }

            for (int i = 0; i < headers.length;
i++) sheet.autoSizeColumn(i);

            try (FileOutputStream fos = new
FileOutputStream(FILE_PATH)) {
                workbook.write(fos);
                System.out.println("Excel файл
создан: " + FILE_PATH);
            }
        } catch (IOException e) {
            System.err.println("Ошибка создания:
" + e.getMessage());
        }
    }

    private static String getCellValue(Cell cell)
{
        if (cell == null) return "";
        return switch (cell.getCellType()) {
            case STRING ->
cell.getStringCellValue();
```

```
            case NUMERIC -> String.valueOf((long)
cell.getNumericCellValue());
            case BOOLEAN ->
String.valueOf(cell.getBooleanCellValue());
            default -> "";
        };
    }
```

Задачи с timus

Задача 1710

task1710.java

```
package lr8.timus;
import java.io.*;
import java.util.*;

public class task1710 {
    public static void main(String[] args) throws
IOException {
        BufferedReader br = new
BufferedReader(new
InputStreamReader(System.in));

        // Читаем точки
        double[] A = readPoint(br);
        double[] B = readPoint(br);
        double[] C = readPoint(br);

        // Вычисляем стороны
        double AB = distance(A, B);
        double BC = distance(B, C);
        double AC = distance(A, C);

        // Находим угол при A (между AB и AC)
        double angleBAC = angleBetween(A, B, C);
```

```
        double angleRad =
Math.toRadians(angleBAC);

        // Пытаемся построить второй треугольник
        // A2 = (0,0), B2 = (AB, 0)
        // C2 лежит на луче под углом angleRad на
расстоянии AC

        double cx = AC * Math.cos(angleRad);
        double cy = AC * Math.sin(angleRad);

        // Проверяем, какая из двух возможных
точек C2 даёт правильное BC
        // Точка выше оси X
        double b2c2_up = distance(new
double[]{AB, 0}, new double[]{cx, cy});
        // Точка ниже оси X
        double b2c2_down = distance(new
double[]{AB, 0}, new double[]{cx, -cy});

        if (Math.abs(b2c2_up - BC) < 1e-9 &&
Math.abs(cy) > 1e-9) {
            // Есть два варианта: cy и -cy
            // Проверяем, совпадают ли
треугольники
            double c1x = cx;
            double c1y = cy;
            double c2x = cx;
            double c2y = -cy;

            // Если треугольники не совпадают
(разные координаты C)
            if (Math.abs(c1y - c2y) > 1e-9) {
                // Проверяем, что C2 не совпадает
с C1 с точностью до поворота
                System.out.println("NO");
                System.out.printf("%.15f
%.15f\n", 0.0, 0.0);
```

```
                System.out.printf("%.15f
%.15f\n", AB, 0.0);
                System.out.printf("%.15f
%.15f\n", cx, -cy);
                return;
            }
        }

        System.out.println("YES");
    }

    static double[] readPoint(BufferedReader br)
throws IOException {
        String[] parts =
br.readLine().trim().split(" ");
        return new double[]{
                Double.parseDouble(parts[0]),
                Double.parseDouble(parts[1])
        };
    }

    static double distance(double[] p1, double[]
p2) {
        double dx = p1[0] - p2[0];
        double dy = p1[1] - p2[1];
        return Math.sqrt(dx*dx + dy*dy);
    }

    static double angleBetween(double[] A,
double[] B, double[] C) {
        double ABx = B[0] - A[0];
        double ABy = B[1] - A[1];
        double ACx = C[0] - A[0];
        double ACy = C[1] - A[1];

        double dot = ABx * ACx + ABy * ACy;
        double cross = ABx * ACy - ABy * ACx;
```

```
        return Math.toDegrees(Math.atan2(cross,
dot));
    }
}
```

Задача 1711

task1711.java

```
package lr8.timus;
import java.io.*;
import java.util.*;

public class task1711 {
    static int n;
    static String[][] variants;
    static int[] order;
    static String[] chosen;

    public static void main(String[] args) throws
IOException {
        BufferedReader br = new
BufferedReader(new
InputStreamReader(System.in));

        // Читаем n
        n = Integer.parseInt(br.readLine());

        // Читаем варианты для каждой задачи
        variants = new String[n][3];
        for (int i = 0; i < n; i++) {
            String[] line = br.readLine().split("
");
            variants[i][0] = line[0];
                variants[i][1] = line[1];
                variants[i][2] = line[2];
```

```
                // Сортируем варианты
лексикографически
                Arrays.sort(variants[i]);
            }

            // Читаем порядок задач
            order = new int[n];
            String[] orderLine =
br.readLine().split(" ");
            for (int i = 0; i < n; i++) {
                order[i] =
Integer.parseInt(orderLine[i]) - 1; // в
0-индексацию
            }

            chosen = new String[n];

            // Запускаем поиск
            if (backtrack(0, null)) {
                for (int i = 0; i < n; i++) {

System.out.println(chosen[i]);
                }
            } else {
                System.out.println("IMPOSSIBLE");
            }
        }

        static boolean backtrack(int pos, String
prevName) {
            if (pos == n) {
                return true;
            }

            int taskIdx = order[pos]; // какая
задача сейчас
```

```
            for (String name : variants[taskIdx])
{
                if (prevName == null ||
name.compareTo(prevName) > 0) {
                    chosen[taskIdx] = name;
                    if (backtrack(pos + 1, name))
{
                        return true;
                    }
                    chosen[taskIdx] = null;
                }
            }

            return false;
        }
    }
```

Вывод:

В ходе лабораторной работы были освоены методы парсинга XML, JSON, HTML и Excel на языке Java. Реализованы программы для чтения, записи, поиска и удаления данных в различных форматах. Использованы библиотеки JAXP, json-simple, Jsoup и Apache POI. Решены задачи Timus №1710 и №1711.